

COMP301

Software Design

Confidence Agent

Static Verification of API Hallucinations In LLM-Generated Code

Progress Report 2

An investigation into how thoughtful software design can improve the trustworthiness of AI systems through benchmark design, experimental methodology, validation architectures, and evidence-based evaluation.

by The Skeptics

University of KwaZulu-Natal

Project Formalities

Domain

Static Verification of API Hallucinations In LLM-Generated Code

Group Name: *The Skeptics*

Group Members:

Member Name	Student Number
Jawad Kachbal	223037045
Joshua Blom	224039813
Mahomed Valli Mahomed	224005878
Andiswa Hlengwa	221002430
Abiel Govender	224058851

Table of Contents

Project Formalities 1

Progress Report 3

1. Executive Summary 3

2. Project Context and Research Question 3

3. Core Definitions 5

4. Scope Decisions and Justification 7

5. The Dataset Specification 8

5.4 Example Task 9

6. The Verification Architecture 12

6.1 The Six Verification Layers 12

6.2 The Evidence Hierarchy 12

6.3 Pylance , Not Proof 13

7. The Confidence Metric (EWCS) 15

7.2 Why Not a Probability? 15

7.5 The EWCS Formula 16

7.6 The Signed Verification Score (Diagnostic) 16

7.8 Sensitivity Analysis (Planned) 17
8. The Decision Policy 18
9. The Agent Controller 19
10. The Experimental Methodology 21
11. Evaluation Metrics 23
12. Current Progress Status 24
13. Next Steps 26
14. Key Decisions Summary 27
15. Glossary 27

Progress Report

COMP301 — Confident Agent

Progress Report 2 - Comprehensive Specification & Design Update

The Skeptics — University of KwaZulu-Natal

---

1. Executive Summary

This project investigates whether an LLM-based code-generation system can use external static verification to reduce unacceptable REST API hallucinations reaching the user, while maintaining appropriate abstention behaviour when verification is inconclusive.

The project has reached a critical milestone: all conceptual, methodological, and specification work is complete. The next phase is implementation.

Project Status at a Glance

Phase	Progress	Status
Phase 1: Conceptual Foundation	100%	Complete
Phase 2: Dataset Specification	100%	Complete
Phase 3: Verification Architecture	100%	Complete
Phase 4: Confidence Metric (EWCS)	100%	Complete
Phase 5: Experimental Design	100%	Complete
Phase 6: Dataset Generation	1%	In Progress
Phase 7: Implementation	0%	Not Started
Phase 8: Experimentation	0%	Not Started
Phase 9: Reporting	0%	Not Started

2. Project Context and Research Question

2.1 Origin

This project originates from the COMP301 Software Design brief on "Confident Agents" - an investigation into how software engineers can design AI systems that recognise the limits of their own knowledge before presenting information to users.

The brief framed the challenge as:

A Confident Agent is not one that is always correct. It is one that knows when it should not be trusted.

This is a philosophy, not just a technical requirement. It reframes the goal of AI safety from "eliminate errors" (impossible) to "recognise and communicate uncertainty" (achievable).

2.2 Why This Matters

Large Language Models have transformed software development. Developers increasingly use LLMs to generate code from natural language descriptions. But LLMs hallucinate - they produce code that looks correct, is presented with confidence, and is actually wrong.

For code generation specifically, hallucinations are particularly dangerous because:

- Code executes. A wrong API call fails or does something unintended.
- Code is trusted. Developers copy-paste without verifying.
- Errors compound. A single wrong function call breaks the whole program.
- Documentation is fragmented. Verifying an API call requires checking multiple sources.

The specific problem we address is REST API hallucinations in generated code.

2.3 The Research Question

After extensive refinement, we settled on:

"Can an LLM-based code-generation system use external static verification to reduce unacceptable REST API hallucinations reaching the user, while maintaining appropriate abstention behaviour when verification is inconclusive?"

Why this wording?

- **"external static verification"** → We do not rely on the LLM to check itself. We use external sources (OpenAPI specs, Pylance) that are independent of the generating model.
- **"reduce unacceptable REST API hallucinations"** → We do not claim to eliminate hallucinations. We measure reduction. "Unacceptable" means claims that violate the API specification. "REST API" scopes the project to a well-defined domain.
- **"reaching the user"** → This is the actual harm we want to prevent. An incorrect claim that never reaches the user is not a failure.
- **"appropriate abstention behaviour"** → The agent must know when to refuse to answer. This is the "confident" part - the agent recognises its own uncertainty.
- **"when verification is inconclusive"** → Not all cases can be resolved. The agent must handle ambiguity.

2.4 The Scientific Hypothesis

Agent-based validation architectures equipped with explicit evidence-based verification and abstention can significantly reduce unacceptable REST API hallucinations compared with an equivalent baseline LLM operating without validation.

**** Research into how exactly we are going to do the statistical analysis is underway – A common reoccurrence so far for our testing is McNemar (pending research) together with the potential variety of stats that can be generated from the structure of our testing.

2.5 The Confident Agent Philosophy

This philosophy guides the entire project:

The agent does not need to be perfect - it needs to recognise the limits of its own knowledge and act accordingly.

The project investigates three capabilities:

1. Detection: Can the agent identify API hallucinations?
2. Correction: Can the agent fix identified hallucinations?
3. Abstention: Can the agent refuse to respond when verification is inconclusive?

3. Core Definitions

3.1 What is a Hallucination?

Definition:

An LLM hallucination is content that appears plausible and confident but is actually incorrect, fabricated, or unsupported by ground truth.

Key characteristics:

Plausible: Looks correct to an untrained eye.
Confident: Presented without hedging or uncertainty.
Incorrect: Violates verified ground truth.

Example:

```
import requests
response = requests.get_json("https://api.github.com/users/octocat")
# ❌ 'get_json' does not exist in the 'requests' library
```

3.2 What is an API?

Definition:

An API (Application Programming Interface) is any documented, programmatically accessible interface - exposed by a language runtime, standard library, third-party framework, operating system, or external web service - that can be invoked by software to perform a specific operation.

Six conceptual layers:

Layer	Description
Owner	Who publishes the interface
Access Mechanism	How it is invoked (import, HTTP verb)
Identifier Path	Fully qualified name (requests.get, /users/{id})
Input Contract	Expected parameters and types
Output Contract	Expected return value or response
Access Point	URL, module, or namespace

3.3 What is an API Hallucination?

Definition:

An API hallucination is a claim in generated code that is VERIFIED INCORRECT against an authoritative ground-truth source.

The key insight: "Cannot be verified" does NOT equal "is false."

We distinguish four states:

State	Meaning	Agent Action
VERIFIED_CORRECT	Evidence supports validity	DELIVER
VERIFIED_INCORRECT	Evidence contradicts validity	CORRECT or ABSTAIN
UNVERIFIABLE	Insufficient evidence	ABSTAIN
NOT_APPLICABLE	Layer irrelevant to this task	Excluded

Critical distinction: NOT_APPLICABLE ≠ UNVERIFIABLE. N/A is excluded from scoring; UNVERIFIABLE counts against confidence.

3.4 The Hallucination Taxonomy (H01–H10)

ID	Type	Layer
H00	No hallucination	N/A
H01	wrong_endpoint	Access Point
H02	wrong_http_method	Access Point
H03	nonexistent_parameter	Input Contract
H04	missing_required_parameter	Input Contract
H05	wrong_parameter_location	Input Contract
H06	wrong_parameter_type	Input Contract
H07	invalid_request_body	Input Contract
H08	missing_required_field	Input Contract
H09	invalid_header_or_authentication	Contextual
H10	nonexistent_api_operation	Identifier

4. Scope Decisions and Justification

4.1 Why Python Only?

- **Reason 1: Pylance —** Microsoft's Pylance is a production-grade static analysis tool for Python. It is free, backed by typeshed (the canonical type stub repository), and accessible programmatically via the Language Server Protocol.
- **Reason 2: requests library —** Python's requests library is the de facto standard for making HTTP requests. It has a simple, uniform API.
- **Reason 3: OpenAPI ecosystem —** The Python ecosystem has mature OpenAPI tooling.
- **Reason 4: Reproducibility —** Python code is easier to reproduce, test, and review.

4.2 Why REST APIs Only?

- **Reason 1: OpenAPI specifications —** REST APIs are described by OpenAPI specifications. These are machine-readable, authoritative, and widely available.
- **Reason 2: Clear verification target —** A REST API claim is easy to verify: does the endpoint exist? Is the method correct? Are the parameters valid?
- **Reason 3: Real-world relevance —** REST APIs are the most common integration point for LLM-generated code.

4.3 Why Static Verification Only?

- **Reason 1: Safety —** Generated code may contain malicious, destructive, or dangerous operations. Static analysis is completely safe.
- **Reason 2: Reproducibility —** Static analysis is deterministic. The same code always produces the same result.
- **Reason 3: No cost, no network dependency —** Static analysis works offline and does not incur API costs.

4.4 Why These Six APIs?

API	Tasks	Why Selected	Auth Type
JSONPlaceholder	150	Simple, no auth	None
GitHub REST API	250	Complex, real	Bearer, OAuth2
Open-Meteo	150	Query params	None (API key opt.)
DummyJSON	150	E-commerce data	None
REST Countries	150	Reference data	API key
Petstore (Swagger)	150	OpenAPI standard	API key, OAuth2

4.5 Why Qwen via LM Studio?

- Reason 1: Free and local
- Reason 2: Complete privacy - no data leaves the machine
- Reason 3: Reproducible with fixed parameters
- Reason 4: OpenAI-compatible API
- Reason 5: Open-source and inspectable

5. The Dataset Specification

5.1 The Master Dataset

Our Master dataset is fixed and contains 1,000 REST API tasks in Python, each with:

- A natural language instruction
- A structured API specification (OpenAPI-derived)
- Ground truth code
- A test case

File format: JSON Lines (JSONL) | File name: rest_api_tasks_v1.jsonl

5.2 Why JSONL Instead of JSON?

Feature	JSONL Advantage
Streaming	Read one task at a time
Appending	Add a line without rewriting the file
Version control	Clean line-by-line diffs
Robustness	A corrupt line doesn't break the file
Memory usage	Only one task in memory at a time

5.3 The Schema

Each task is a JSON object with these fields:

Field	Type	Purpose
task_id	string	Unique identifier
**language	string	"python"
natural_language_instruction	string	The prompt
api_specification	object	API details
ground_truth	object	Correct code and metadata
test_case	object	Test inputs and outputs
metadata	object	Context and tags

```
{
  "task_id": "REST_PY_JP_0001",
  "language": "python",
  "natural_language_instruction": "Write a Python function that fetches the
    details of a specific user from the JSONPlaceholder API.",
  "api_specification": {
    "api_name": "JSONPlaceholder",
    "base_url": "https://jsonplaceholder.typicode.com",
    "endpoint": "/users/{user_id}",
    "method": "GET",
    "parameters": [
      {"name": "user_id", "in": "path", "type": "integer",
       "required": true, "description": "The user's ID."}
    ],
    "headers": [],
    "request_body": null,
    "response_schema": {...},
    "auth": null,
    "openapi_url": "https://jsonplaceholder.typicode.com/openapi.json"
  },
  "ground_truth": {
    "code": "import requests\n\ndef get_user(user_id: int) -> dict:\n    ...",
    "expected_behavior": "Fetches a single user by ID.",
    "verified_by": "manual_review_against_openapi",
    "verification_date": "2026-09-01"
  },
  "test_case": {
    "inputs": {"user_id": 1},
    "expected_output": {...}
  },
  "metadata": {
    "api_name": "JSONPlaceholder",
    "category": "read",
    "difficulty": "easy",
    "source_openapi": "https://jsonplaceholder.typicode.com/openapi.json",
    "source_docs": "https://jsonplaceholder.typicode.com/guide/",
    "created_date": "2026-09-01",
    "reviewed_by": "manual_review",
    "tags": ["rest", "json", "user", "get"]
  }
}
```

5.4 The Evaluation Dataset

Field	Type	Purpose
task_id	string	Unique identifier
**language	string	"python"
natural_language_instruction	string	The prompt

***This attribute may be removed. The idea is that the API Verification Layer and the API Invoked Language Verification layer (ie Pylance for Python) is separate from each other, which will allow us to potentially expand to other languages like JavaScript, Java ect.*

This test dataset omits other labels, leaving only the task_ID (for tracking) and the actual prompt itself. The idea is to have the baseline and our Agentic Pipeline perform a "*Blind Test*" using the Test Dataset. The outputs of each will be track and evaluated later on.

The separation is:

- Master Dataset = WHAT IS CORRECT
- Evaluation Dataset = WHETHER THE MODEL'S OUTPUT IS CORRECT

5.6 Composition

API distribution:

- JSONPlaceholder — 150 tasks
- GitHub REST API — 250 tasks
- Open-Meteo — 150 tasks
- DummyJSON — 150 tasks
- REST Countries — 150 tasks
- Petstore (Swagger) — 150 tasks

TOTAL — 1,000 tasks

Operation distribution:

- GET (list) — ~25%
- GET (single resource) — ~30%
- POST (create) — ~15%
- PUT / PATCH (update) — ~15%
- DELETE — ~10%
- Search / Filter — ~5%

Difficulty distribution:

- Easy — ~40%
- Medium — ~40%
- Hard — ~20%

Authentication distribution:

- None — ~55%
- API Key — ~15%
- Bearer Token — ~20%
- OAuth2 — ~10%

5.7 Why Neutral Prompts?

All prompts are written in neutral natural language. They do not contain:

- "Use requests.get()"
- "Make sure the endpoint is /users/{id}"
- "Avoid hallucinating"

They DO contain:

> "Write a Python function that fetches a user from the JSONPlaceholder API"

If we tell the model to "use requests.get()", we are testing whether the model can follow instructions, not whether it can generate correct code. This was found to contribute negatively during earlier draft datasets.

6. The Verification Architecture

6.1 The Six Verification Layers

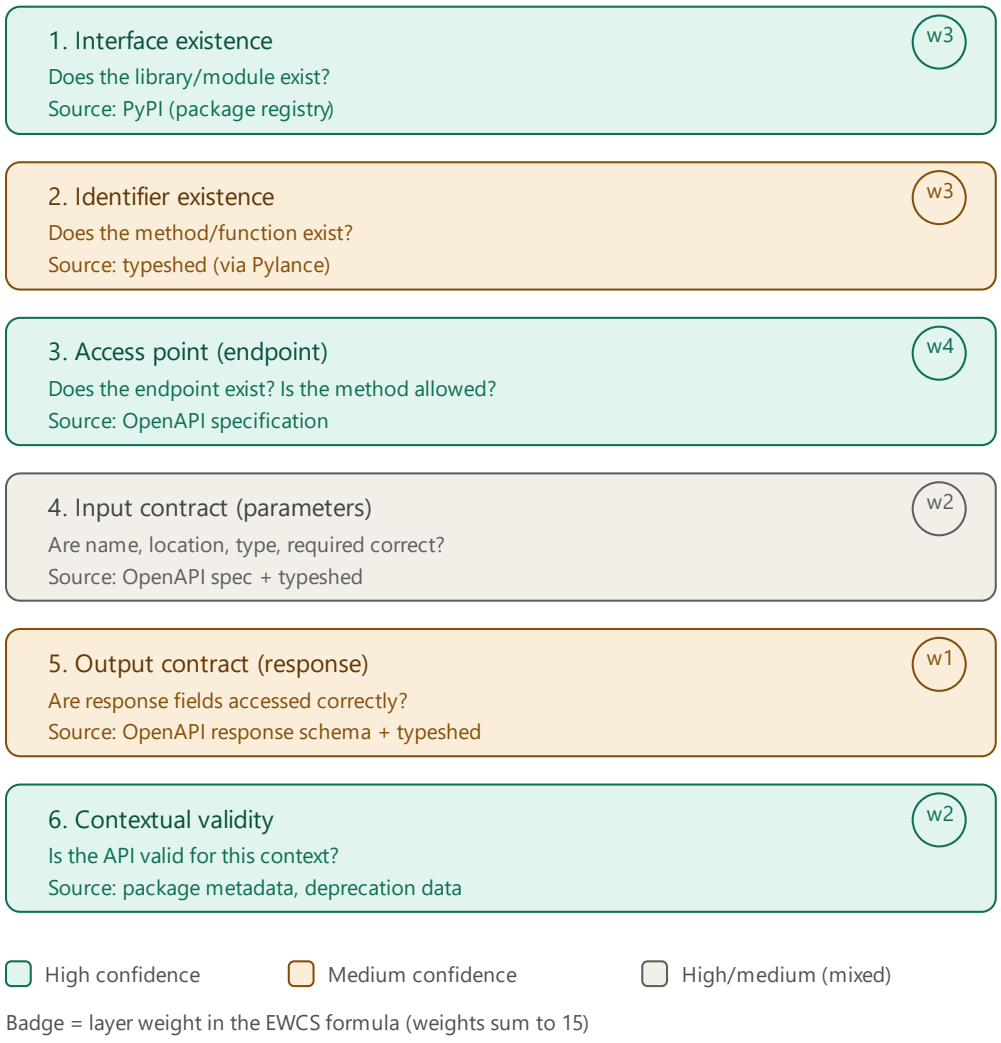
Layer	Question	Source	Confidence
1. Interface Existence	Does the library/module exist?	PyPI (package registry)	HIGH
2. Identifier Existence	Does the method/function exist?	typeshed (via Pylance)	MEDIUM
3. Access Point (Endpoint)	Does the endpoint exist? Is the method allowed?	OpenAPI specification	HIGH
4. Input Contract (Parameters)	Are parameters correct (name, location, type, required)?	OpenAPI spec + typeshed	HIGH / MEDIUM
5. Output Contract (Response)	Are response fields accessed correctly?	OpenAPI response schema + typeshed	MEDIUM
6. Contextual Validity	Is the API valid for this context?	Package metadata, platform detection, deprecation data	HIGH

6.2 The Evidence Hierarchy

Tier	Factor	Sources
Tier 1 — AUTHORITATIVE	1.00	Official OpenAPI specifications; official API documentation
Tier 2 — STRONG SUPPORTING	0.80	Official package registries (PyPI); type definitions (typeshed)
Tier 3 — EMPIRICAL	0.90	Empirical execution (reserved for future work; NOT used in this phase)
Tier 4 — SUPPORTING	0.50	Pylance diagnostics

Evidence tier is claim-dependent. Tier 3 is not universally weaker than Tier 2.

6.3 Verification Flow



6.4 Pylance

Pylance is a static analysis tool. Its diagnostics are strong evidence but not definitive proof. Reasons Pylance may fail to resolve a valid attribute:

- Dynamic attributes (__getattr__, __setattr__)
- Metaclasses and generated methods
- Runtime registration or plugins
- Incomplete or outdated type stubs
- Conditional imports or environment-specific code

Therefore, Pylance diagnostics is treated as strong evidence, not definitive proof.

6.5 Why Not Runtime Execution?

We do not execute generated code. Reasons:

- Safety: Never execute untrusted code.
- Reproducibility: Same code always produces same result.
- Cost: No API charges.
- Determinism: No network variability.

The trade-off: we cannot catch every error. But for API hallucinations specifically, static verification catches the vast majority.

6.6 The MCP Tool Attachment

The idea is to have The Confident Agent is exposed as an MCP (Model Context Protocol) tool, allowing it to attach to any MCP-compatible LLM:

```
{
  "tool": "verify_api_usage",
  "description": "Verifies REST API usage. Returns verification state and
                  evidence-weighted confidence.",
  "input": {
    "code": "string",
    "language": "string"
  },
  "output": {
    "verification_state": "VERIFIED_CORRECT | VERIFIED_INCORRECT |
                           UNVERIFIABLE",
    "ewcs": "float [0,1]",
    "signed_score": "float [-1,1]",
    "evidence": [...],
    "decision": "DELIVER | DELIVER_WITH_FLAG | CORRECT | ABSTAIN"
  }
}
```

7. The Confidence Metric (EWCS)

7.1 Terminology

Three concepts are explicitly separated:

Term	Definition
Verification State	Categorical result per claim
Evidence-Weighted Confidence Score (EWCS)	Primary metric [0, 1]
Signed Verification Score	Diagnostic metric [-1, 1]

Important distinction: Ground truth = the dataset specification (known correct API usage). Verification state = the verifier's estimate of correctness.

The verifier's output is NOT ground truth. It is an estimate of correctness relative to ground truth.

7.2 Why Not a Probability

The EWCS is NOT a probability. A score of 0.95 does not mean "95% probability of being correct." It means "95% of applicable verification weight is supported by positive evidence."

The Evidence-Weighted Confidence Score is an evidence-weighted measure of verification strength, not a calibrated probability of correctness.

7.3 Layer Weights

Layer	Weight
1. Interface Existence	3
2. Identifier Existence	3
3. Access Point (Endpoint)	4
4. Input Contract (Parameters)	2
5. Output Contract (Response)	1
6. Contextual Validity	2
TOTAL	15

The weights are operational design choices, not empirically established universal weights.
****Sensitivity analysis is planned.*

7.4 Tier Factors

Tier	Source	Factor
Tier 1	OpenAPI, official documentation	1.00
Tier 2	Package registry, typeshed	0.80
Tier 3	Empirical execution (reserved)	0.90
Tier 4	Pylance diagnostics	0.50
NONE	No evidence	0.00

7.5 The EWCS

EWCS is the proportion of applicable verification weight supported by positive evidence:

$$EWCS = \frac{\Sigma\ (\ w_l \times tier_factor_l\ )\ \ for\ state = VERIFIED_CORRECT}{\Sigma\ (\ w_l\ )\ \ for\ applicable\ layers}$$

where EWCS lies in [0, 1] with:

- w_l = The layer Weight (*see 7.3 above)*
- tier_factor = Evidence Strength (*see 7.4 above)*
- $\Sigma\ (\ w_l \times tier_factor_l\ )$ = The sum of the product of w_l and the tier_factor.
- $\Sigma\ (\ w_l\ )$ = The sum of the total weights.

Interpretation:

- 1.00 → All applicable weight supported by Tier 1 positive evidence
- 0.80 → All applicable weight supported by Tier 2 positive evidence
- 0.50 → Half of applicable weight supported
- 0.00 → No positive evidence found

7.6 The Signed Verification

It should be noted that the EWCS only counts positive evidence. As a result, we cannot distinguish between:

- A layer that is UNVERIFIABLE (we don't know)
- A layer that is VERIFIED INCORRECT (we know its wrong)

This is where Signed Verification comes in. ***Signed Verification*** is a ***diagnostic*** metric in the range [-1,1] captures both positive and contradictory evidence across the six verification layers – a secondary internal confidence metric used for analysis. The signed score distinguishes the above mentioned cases by giving negative scores to incorrect layers.

Signed score lies in [-1, 1]:

$$signed_score = \frac{\Sigma\ (\ w_l \times s_l\ )}{\Sigma\ (\ w_l\ )}$$

$$l \in applicable\ layers$$

where s_l:

- VERIFIED_CORRECT → +1.0 × tier_factor
- VERIFIED_INCORRECT → −1.0 × tier_factor
- UNVERIFIABLE → 0.0
- NOT_APPLICABLE → excluded

The signed score is NOT reported in the primary results. It is a diagnostic metric for internal analysis and debugging

7.7 Worked Example

Task: Fetch a user by ID from JSONPlaceholder.

LLM generates (incorrect):

```
response = requests.get(f"https://jsonplaceholder.typicode.com/user/{user_id}")
```

Note: /user/{user_id} is incorrect (correct is /users/{user_id}).

Layer	Weight	State	Tier	Factor	Positive Weight
1. Interface	3	CORRECT	2	0.80	2.40
2. Identifier	3	CORRECT	2	0.80	2.40
3. Access Point	4	INCORRECT	1	1.00	0.00
4. Input Contract	2	CORRECT	1	1.00	2.00
5. Output Contract	1	CORRECT	2	0.80	0.80
6. Contextual	2	N/A	—	—	excluded

```
EWCS = (2.40 + 2.40 + 0.00 + 2.00 + 0.80) / (3 + 3 + 4 + 2 + 1)
     = 7.60 / 13
     = 0.585

Decision: Safety constraint fires (Layer 3 INCORRECT) → CORRECT.
```

For the correct version of the code:

```
EWCS = (2.40 + 2.40 + 4.00 + 2.00 + 0.80) / 13
     = 11.60 / 13
     = 0.892

Decision: No incorrect claims; EWCS ≥ 0.85 → DELIVER.
```

Scheme	Weights (L1–L6)	Thresholds (Deliver/Flag)
A. Proposed	3, 3, 4, 2, 1, 2	0.85 / 0.60
B. Equal	1, 1, 1, 1, 1, 1	0.85 / 0.60
C. Endpoint-Heavy	2, 2, 6, 2, 1, 2	0.85 / 0.60
D. Strict Thresholds	3, 3, 4, 2, 1, 2	0.90 / 0.70
E. Lenient Thresholds	3, 3, 4, 2, 1, 2	0.80 / 0.50

If the intervention reduces hallucinations under all five schemes, the result is robust.

8. The Decision Policy

8.1 Three Components

The decision policy has three distinct components:

```
Component 1: Safety Constraint (independent of confidence)

  IF any layer is VERIFIED_INCORRECT:
    → emit CORRECT (route to correction sub-loop)

Component 2: Confidence Thresholds

  ELSE IF EWCS >= 0.85 → emit DELIVER
  ELSE IF EWCS >= 0.60 → emit DELIVER_WITH_FLAG
  ELSE                 → emit ABSTAIN
```

The safety constraint is NOT a confidence rule. It is a hard rule that fires independently of EWCS. This ensures that a high-confidence code with a single critical error is still caught.

8.2 Why the Safety Constraint Matters

In the incorrect JSONPlaceholder example, EWCS = 0.585. Without the safety constraint, this would trigger ABSTAIN - the correct outcome, but for the wrong reason.

The safety constraint ensures the decision is driven by the presence of a critical error, not by the numeric score alone.

8.3 Decision Thresholds (Researcher-Defined)

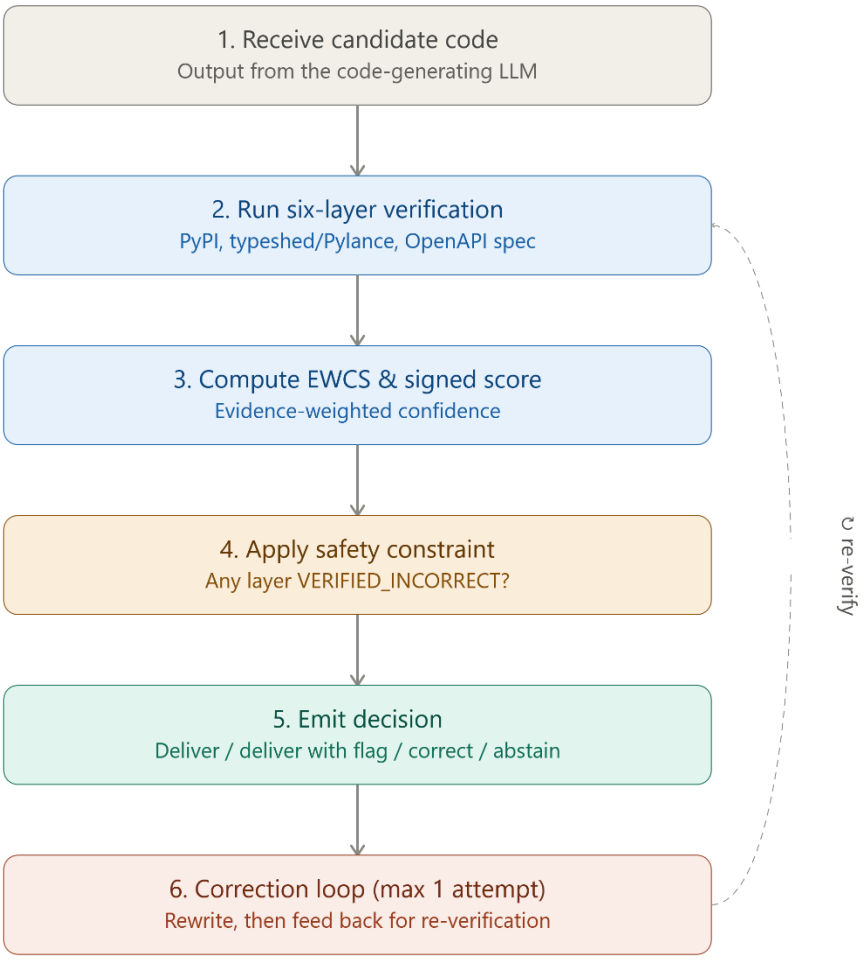
Threshold	Value	Role
High confidence	0.85	Deliver without flag
Moderate confidence	0.60	Deliver with flag
Below moderate	< 0.60	Abstain

Thresholds are researcher-defined. Sensitivity analysis is planned.

9. The Agent Controller

9.1 Correction Sub-Loop

Correction is NOT part of the decision function. It is orchestrated by the agent controller.



If step 4 finds no error: skip to step 5 without looping through step 6

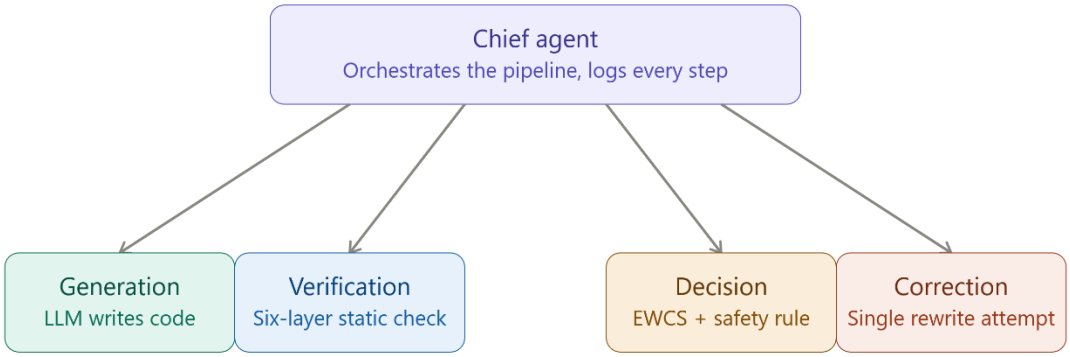
Maximum correction attempts: 1 (fixed to avoid sampling bias).

9.2 Why the Separation?

Property	Reason
Purity	The decision function has no side effects.
Testability	The decision logic can be tested in isolation.
Modularity	Correction logic can be swapped or extended.
Auditability	Every step is logged separately.
Alignment	Matches the brief's "Chief Agent coordinating specialized agents" pattern.

9.3 Multi-Agent Reframing

To align with the brief's suggestion of a multi-agent architecture, the pipeline can be reframed as cooperating agents:



Each sub-agent reports back to the chief agent; correction results are re-checked by verification

This satisfies the brief's suggestion of multi-agent design.

10. The Experimental Methodology

10.1 Experimental Design

We use a controlled experiment comparing two conditions:

BASELINE

Raw LLM generates code from the natural language instruction. No verification. No correction. No abstention. Output: Generated code.

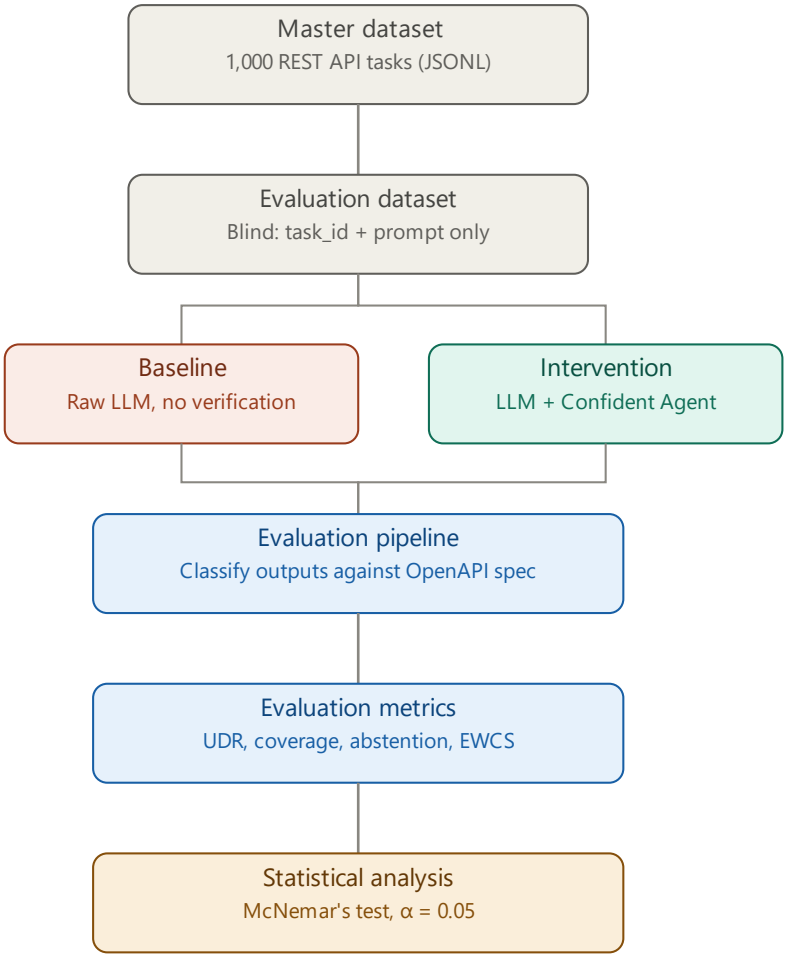
INTERVENTION

LLM generates code, then the Confident Agent:

1. Verifies the code statically
2. Corrects it if incorrect (max 1 attempt)
3. Abstains if verification is inconclusive

Output: Verified code OR abstention

10.2 Experimental Flow



10.3 Generation Parameters (Fixed)

Parameter	Value
Model	Qwen3.5-9B-Instruct
Temperature	0.3
Top-p	0.9
Max Tokens	4096
Context Length	8192
Seed	Fixed (where supported)
System Prompt	"You are a code generation assistant. Provide only code."

10.4 The Evaluation Pipeline

For each output (baseline and intervention):

1. Parse the generated code
2. Extract the API call
3. Compare against api_specification (OpenAPI)
4. Classify outcome: Correct (matches OpenAPI spec) / Hallucination (violates spec, H01-H10) / Python Error (syntax or type error unrelated to API) / Ambiguous (cannot determine)
5. For hallucinations, record the specific H-code
6. For interventions, check if the agent detected/corrected the issue

11. Evaluation Metrics

11.1 Primary Metric

UNSAFE DELIVERY RATE (UDR):

= Incorrect outputs delivered to the user / Total outputs

This directly measures the harm we want to prevent.

11.2 Secondary Metrics

Metric	Definition
Coverage	Delivered / Total
Abstention Rate	Abstained / Total
Selective Accuracy	Correct Delivered / Total Delivered
Correction Success Rate	Successful corrections / Attempts
Correction Regression	Correct → Incorrect after intervention
EWCS Distribution	Mean/median EWCS (baseline vs intervention)
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1 Score	2 * (P * R) / (P + R)

11.3 The Four Evaluation Questions

- **Q1 — Does the LLM hallucinate?** Measured by natural hallucination rate (baseline). Purpose: establishes the problem.
- **Q2 — Can the verifier detect hallucinations?** Measured by Precision, Recall, F1. Purpose: evaluates verifier capability.
- **Q3 — Does the complete agent reduce bad outputs?** Measured by Unsafe Delivery Rate (baseline vs intervention). Purpose: primary experimental result.
- **Q4 — Does it know when not to answer?** Measured by Abstention Rate, Selective Accuracy, Coverage. Purpose: evaluates the Confident Agent philosophy.
- **Q5 — Does confidence improve?** Measured by EWCS distribution (baseline vs intervention). Purpose: secondary quantitative metric.

11.4 ******Statistical Analysis - *(Pending research)*

Statistical test	McNemar's test for paired binary outcomes
Primary outcome	Unacceptable API claims reaching the user per prompt (binary: Yes/No)
Significance level	$\alpha = 0.05$
Effect size	Odds ratio or relative risk reduction with 95% CI

The contingency table (McNemar's test compares cells b and c):

Baseline \ Intervention	Correct	Wrong
Correct	a	b
Wrong	c	d

12. Current Progress Status

12.1 Completed Work

✅ **Research Definition**

- Research question finalized
- Scientific hypothesis finalized
- Confident Agent philosophy articulated

✅ **Definitions**

- Hallucination defined
- API defined (six conceptual layers)
- API hallucination defined (four-state model)
- Hallucination taxonomy defined (H01–H10)

✅ **Scope Decisions**

- Python only
- REST APIs only
- Static verification only
- Six APIs selected
- Qwen via LM Studio

✅ **Dataset Specification**

- JSONL format chosen
- Schema finalized (7 top-level fields)
- Composition defined (1,000 tasks, distribution)
- Quality criteria defined
- Sample tasks drafted (11 tasks)

✅ **Verification Architecture**

- Six-layer model finalized
- Evidence hierarchy defined
- Decision engine (four-state) designed
- Correction loop designed
- Abstention mechanism designed
- Pylance integration designed
- MCP tool designed

✅ **Confidence Metric**

- Terminology finalized (EWCS, Signed Score, Verification State)
- Layer weights defined
- Tier factors defined
- Formula finalized
- Worked examples completed
- Sensitivity analysis planned

✅ **Experimental Methodology**

- Baseline vs intervention design finalized
- Generation parameters fixed
- Evaluation pipeline designed
- Metrics defined
- Statistical analysis planned

12.2 In Progress

🔨 **Dataset Generation**

- In progress

🔨 **OpenAPI Spec Collection**

- JSONPlaceholder spec available
- GitHub spec available (needs curation)
- Petstore spec available
- REST Countries spec available
- Open-Meteo spec needs manual creation
- DummyJSON spec needs manual creation

12.3 Not Started

☐ **Implementation**

- Pylance LSP wrapper
- OpenAPI validator
- Evidence aggregation engine
- Decision engine (four-state + EWCS)
- Correction loop
- MCP tool

☐ **Experimentation**

- Baseline run (1,000 tasks)
- Intervention run (1,000 tasks)
- Evaluation
- Statistical analysis

☐ **Reporting**

- Analysis of results
- Final report
- Presentation

13. Next Steps

13.1 Immediate Actions

1. Confirm API selection and task distribution
2. Resolve REST Countries API key handling
3. Decide on OpenAPI spec approach for Open-Meteo and DummyJSON
4. Download/curate OpenAPI specs for all six APIs

13.2 Short-Term Actions

5. Generate remaining dataset tasks
6. Run automated dataset validation
7. Conduct 10% manual review of the dataset.

13.3 Medium-Term Actions

8. Build Pylance LSP wrapper
9. Build OpenAPI validator
10. Build evidence aggregation and decision engine
11. Build correction loop
12. Build MCP tool

13.4 Long-Term Actions

13. Run baseline experiment (1,000 tasks)
14. Run intervention experiment (1,000 tasks)
15. Run evaluation pipeline
16. Perform statistical analysis
17. Perform sensitivity analysis (5 schemes)
18. Write final report
19. Prepare presentation

14. Key Decisions Summary

Decision	Rationale
Python only	Pylance, requests library, OpenAPI ecosystem
REST APIs only	OpenAPI specifications available
Static verification only	Safety, reproducibility, cost
Six APIs (JSONPlaceholder, GitHub, Open-Meteo, DummyJSON, REST Countries, Petstore)	Diversity of domain, complexity, authentication
Qwen via LM Studio	Free, local, reproducible
JSONL format	Streaming, appendable, VC-friendly
Dataset = ground truth only	Neutral, reusable across conditions
Four-state verification model	Distinguishes N/A from UNVERIFIABLE
EWCS as primary confidence metric	Interpretable [0,1] range; proportion of positive evidence
Safety constraint separate from confidence thresholds	Ensures critical errors override high confidence
Correction outside decide()	Pure classifier; controller orchestrates correction
One correction attempt	Avoids sampling bias
McNemar's test	Appropriate for paired binary outcomes
MCP tool attachment	Standard protocol for LLM tool integration
Sensitivity analysis (5 schemes)	Tests robustness to weight and threshold choices

15. Glossary

Term	Definition
Abstention	The agent's refusal to deliver code when verification is inconclusive.
Baseline	The experimental condition where the raw LLM generates code without verification.
Confident Agent	An agent that recognises the limits of its own knowledge and abstains when verification is inconclusive.
Correction Loop	The process of feeding validation errors back to the LLM for a single rewrite attempt.
Evidence Hierarchy	The ranking of evidence sources by authority (Tier 1 = authoritative, Tier 4 = supporting).
EWCS (Evidence-Weighted Confidence Score)	The primary confidence metric. Proportion of applicable weight supported by positive evidence. Range [0, 1]. Not a probability.
Ground Truth	The correct API usage recorded in the dataset specification. Not the verifier's output.
H01–H10	The ten REST-specific hallucination types.
JSONL	JSON Lines - one JSON object per line.
McNemar's Test	A statistical test for paired binary outcomes.
MCP	Model Context Protocol - a standard for LLM tool integration.
OpenAPI	A specification standard for describing REST APIs.

Pylance	Microsoft's Python language server, used for static analysis.
Signed Verification Score	Diagnostic metric in [-1, 1]. Captures both positive and contradictory evidence. Not reported in primary results.
Three-State (Four-State) Model	The classification of API claims as VERIFIED_CORRECT, VERIFIED_INCORRECT, UNVERIFIABLE, or NOT_APPLICABLE.
Unsafe Delivery Rate (UDR)	The proportion of outputs that contain an incorrect API claim and are delivered to the user.
Verification State	The per-layer categorical result from the verifier. NOT ground truth - an estimate of correctness relative to ground truth.